
aliases: - 25-function-basics

函数基础：定义 / 参数 / 返回值

前置：已掌握变量和条件语句 目标：能独立写出带参数和返回值的函数 用时：约 12 分钟

相关笔记：[递归函数](#) · [闭包与装饰器](#) · [作用域规则](#) · [args · *kwargs](#) · [课堂老师函数程序对照](#)

一、为什么要用函数

```
# 没有函数：代码重复
print("欢迎张三登录")
# ... 50 行逻辑 ...
print("欢迎李四登录")
# ... 同样的50行逻辑 ...
print("欢迎王五登录")
# ... 同样的50行逻辑 ...

# 有函数：写一次，用无数次
def welcome(name):
    print(f"欢迎 {name} 登录")
    # ... 50 行逻辑 ...

welcome("张三")
welcome("李四")
welcome("王五")
```

函数的好处：消除重复、逻辑复用、方便修改。

二、定义和调用

```
# 定义
def greet():
    print("Hello!")

# 调用
greet()    # Hello!
```

结构：

```
def 函数名(参数列表):
    """文档字符串（可选）"""
    函数体
    return 返回值（可选）
```

规则： - `def` 关键字开头 - 函数名 + 圆括号 + 冒号 - 函数体**缩进** - 参数和 `return` 都是可选的

三、参数

位置参数（最常用）

```
def greet(name, greeting):
    print(f"{greeting}, {name}")

greet("张三", "你好")    # 你好, 张三
greet("李四", "Hello")   # Hello, 李四
```

参数的顺序很重要，按位置一一对应。

关键字参数

```
# 调用时指名道姓，顺序可以乱
greet(greeting="你好", name="张三")    # 你好, 张三
```

默认参数（缺省参数）

```
def greet(name, greeting="你好"):  
    print(f"{greeting}, {name}")  
  
greet("张三")           # 你好, 张三 — 使用默认值  
greet("李四", "Hello")  # Hello, 李四 — 覆盖默认值
```

默认参数陷阱：默认值只在定义时计算一次！

```
def add_item(item, lst=[]):  # ❌ 坑！  
    lst.append(item)  
    return lst  
  
print(add_item(1))  # [1]  
print(add_item(2))  # [1, 2] — 不是预期的 [2]！  
  
# ✅ 正确做法：用 None 做默认值  
def add_item(item, lst=None):  
    if lst is None:  
        lst = []  
    lst.append(item)  
    return lst
```

四、返回值

```
# 返回一个值  
def add(a, b):  
    return a + b  
  
result = add(3, 5)  
print(result)  # 8  
  
# 返回多个值（实际上返回元组）  
def get_min_max(nums):  
    return min(nums), max(nums)  
  
result = get_min_max([3, 1, 4, 1, 5])  
print(result)  # (1, 5)
```

```
print(type(result))    # <class 'tuple'>

# 解包接收
min_val, max_val = get_min_max([3, 1, 4, 1, 5])
print(f"最小值: {min_val}, 最大值: {max_val}")

# 不写 return → 返回 None
def say_hello(name):
    print(f"你好, {name}")

result = say_hello("张三")    # 你好, 张三
print(result)                 # None
```

五、文档字符串 (docstring)

```
def calculate_bmi(weight, height):
    """计算BMI指数

    参数:
        weight: 体重 (公斤)
        height: 身高 (米)

    返回:
        BMI 值 (float)
    """
    return weight / (height ** 2)

# 查看文档
help(calculate_bmi)    # 打印上面的 docstring
print(calculate_bmi.__doc__)    # 直接获取文档文本
```

团队协作必写 docstring，考试和面试也会加分。

六、类型提示 (Type Hints)

1. 基础语法

```
# 变量注解: 变量名: 类型 = 值
name: str = "张三"
age: int = 20

# 函数参数注解: 参数名: 类型
# 返回类型注解: -> 类型
def greet(name: str, age: int) -> str:
    return f"{name} 今年 {age} 岁"

print(greet("张三", 20)) # 张三 今年 20 岁
```

写法: **变量名在前, 冒号, 类型**。 `变量名: 类型` 读作"这个变量的类型是"。

```
price: float = 99.8 # 浮点数
is_active: bool = True # 布尔值
tags: list[str] = ["a", "b"] # 字符串列表
```

2. 联合类型: 一个字段可能是多种类型

```
# str | None = "可以是字符串, 也可以是 None (空)"
content: str | None = None

# 等价的老写法:
from typing import Optional
content: Optional[str] = None
```

"为什么类型写在变量名后面?"

Python 语法设计就是 `变量名: 类型`, 不是 `类型 变量名` (后者是 Java/C 的写法)。

Java: `String name = "张三"` Python: `name: str = "张三"`

读法一样: "**name 这个变量, 类型是 str, 值是'张三'**".

3. 为什么类型注解很重要

```
# 没有类型注解：你猜这个函数需要什么参数？
def process(data, config):
    pass

# 有类型注解：一眼就知道要什么
def process(data: list[int], config: dict[str, str]) -> bool:
    pass
```

类型注解的好处： 1. **IDE 自动补全**（输入 `data.` 会弹出列表方法） 2. **代码自文档化**（不用看函数体就知道参数要求） 3. **Pydantic 用类型做校验**（`content: str` → 自动校验传进来的是不是字符串） 4. **mypy 静态检查**（在运行前发现类型不匹配）

4. 常见类型写法速查

```
# 基础类型
name: str = "abc"
count: int = 42
ratio: float = 3.14
done: bool = True

# 容器类型（Python 3.9+ 可以直接用，不用从 typing 导入）
items: list[int] = [1, 2, 3]
mapping: dict[str, int] = {"a": 1}
unique: set[str] = {"x", "y"}
pair: tuple[str, int] = ("abc", 42)

# 联合类型（Python 3.10+）
maybe: str | None = None
value: int | str = 42 # 可以是整数，也可以是字符串

# 在 Pydantic 中的实际应用
class NoteCreate(BaseModel):
    content: str # 必填，必须是字符串

class NoteUpdate(BaseModel):
    content: str | None = None # 可选，可以是字符串或空
```

5. 重要提醒

类型注解**不会影响运行**——即使你标注 `name: str` 然后赋值数字，Python 也不会报错。它是给**人**和**工具**看的约定。

七、函数是第一公民

```
# 函数可以赋值给变量
def square(x):
    return x ** 2

f = square          # f 现在是 square 函数的别名
print(f(5))         # 25

# 函数可以作为参数传递
def apply(func, value):
    return func(value)

result = apply(square, 4)
print(result)       # 16
```

总结

概念	写法	说明
定义	<code>def 函数名():</code>	def + 函数名 + 冒号
参数	<code>def f(x, y):</code>	位置参数按顺序传
关键字参数	<code>f(y=1, x=2)</code>	调用时指名
默认参数	<code>def f(x=10):</code>	参数有默认值
返回值	<code>return 值</code>	可返回多个（元组）
无返回值	不写 return	返回 None
文档	<code>"""说明"""</code>	函数体第一行

概念	写法	说明
类型提示	<code>def f(x: int) -> str:</code>	代码更清晰

备考相关

- [[EXAM_PREP/day01/00_今日任务]] — Day 1 基础语法（预热）
- [[EXAM_PREP/day03/00_今日任务]] — Day 3 函数专项
- [[EXAM_PREP/day06/00_今日任务]] — Day 6 老师课堂代码重放
- [[EXAM_PREP/day07/00_今日任务]] — Day 7 学生管理系统综合题